

Linux Commands Cheatsheet

OS with Linux

Syntax, key options, and examples for every command in the source set

Contents

1	1. Navigation: pwd, cd, ls, mkdir	3
1.1	1.1 pwd — print working directory	3
1.2	1.2 cd — change directory	3
1.3	1.3 ls — list directory contents	3
1.4	1.4 mkdir — make directories	3
2	2. Creating and Viewing Files: touch, cat, head, tail, echo, wc	4
2.1	2.1 touch — create empty files, update timestamps	4
2.2	2.2 cat — view, concatenate, create	4
2.3	2.3 head — first lines	4
2.4	2.4 tail — last lines	4
2.5	2.5 echo — print text	5
2.6	2.6 wc — count lines, words, bytes	5
3	3. Copy, Move, Delete: cp, mv, rm	5
3.1	3.1 cp — copy files and directories	5
3.2	3.2 mv — move and rename	5
3.3	3.3 rm — remove files and directories	6
4	4. Help: man	6
4.1	4.1 man — read manual pages	6
5	5. Text Processing: cut, paste, sort, grep, locate	6
5.1	5.1 cut — slice columns out of lines	6
5.2	5.2 paste — merge files side by side	6
5.3	5.3 sort — sort lines	7
5.4	5.4 grep — search text by pattern	7
5.5	5.5 locate — find files by name	7
5.6	5.6 Piping text tools together	7
6	6. Permissions and Privilege: chmod, sudo	8
6.1	6.1 Reading permissions	8
6.2	6.2 chmod — change mode	8
6.3	6.3 sudo — run with elevated privilege	8
7	7. Archives and Compression: tar, zip, gzip, bzip2, xz	8
7.1	7.1 tar — tape archive	8
7.2	7.2 zip, gzip, bzip2, xz	9
8	8. Disks, Memory, System: df, du, free, top, vmstat, uname	9
8.1	8.1 df — disk free	9
8.2	8.2 du — disk usage	9
8.3	8.3 free — memory overview	10
8.4	8.4 top — live process monitor	10
8.5	8.5 vmstat — virtual memory statistics	10
8.6	8.6 uname — system information	10

9	9. Processes: fork, exec, ps, kill, nice	10
9.1	9.1 fork() — create a process	10
9.2	9.2 exec() family — run another program	11
9.3	9.3 ps — process status	11
9.4	9.4 kill — signal processes	11
9.5	9.5 nice — process priority	11
10	10. Master Quick Reference	11

1. Navigation: pwd, cd, ls, mkdir

1.1 pwd — print working directory

Prints the absolute path of the current directory.

```
pwd
pwd -L      # logical path: as navigated, keeps symbolic links
pwd -P      # physical path: resolves symbolic links to the real location
```

1.2 cd — change directory

With no argument (or ~) it returns to the home directory.

```
cd [directory]
cd Documents          # into a subdirectory
cd /var/log           # absolute path
cd ~                  # home (bare cd does the same)
cd ..                 # parent directory, one level up
cd -                  # previous directory you were in
cd /                  # filesystem root
cd "My songs"         # quoted name with spaces (or: cd My\ songs)
cd Documents/geeksforgeeks/example # nested path in one step
cd /var/l + Tab       # tab-completion of directory names
```

cd .. = parent, cd - = previous location, cd ~ = home, cd / = filesystem root. Verify where you landed with pwd; chain with cd /path && ls; shortcut repeated jumps with alias docs="cd ~/Documents" in ~/.bashrc.

1.3 ls — list directory contents

Lists files and directories; default order is alphabetical.

```
ls [option] [file/directory]
ls -l          # long format: permissions, owner, size, date
ls -lh         # long format, human-readable sizes (1K, 234M, 2G)
ls -a         # all files, including hidden dot-files
ls -t | head -1 # most recently modified file
ls -lt        # long format sorted by time, newest first
ls -ltr       # oldest first (last-edited file on the last line)
ls -l         # one entry per line
ls -R /etc/apt # recursive listing through subdirectories
ls -i         # show inode number of each file
ls -ld /etc   # info about the directory itself, not its contents
ls -F         # classify: / directory, * executable, @ link
ls -n         # numeric UID/GID instead of names
```

Option	Meaning
-l	Long format: permissions, links, owner, group, size, date, name.
-a / -A	All files including hidden ones (-A skips . and ..).
-h	Human-readable sizes; almost always combined as ls -lh.
-t	Sort by modification time, newest first.
-r	Reverse the order.
-S	Sort by size, largest first.
-R	Recurse into subdirectories.
-1	One entry per line.
-i	Show the inode number.
-d	List the directory itself, not its contents.
-F / --color=auto	Classify entries with symbols / colours.

Reading ls -l output, field by field: type character (- regular file, d directory, l symlink, s socket), 9 permission characters (owner/group/others as rwx triples), link count, owner, group, size in bytes, modification date, file name.

1.4 mkdir — make directories

Creates directories; silent on success, so verify with ls or use -v.

```
mkdir [options] dir_name
mkdir Linux          # single directory
mkdir dir1 dir2 dir3 # several at once
```

```
mkdir dir{1..15}           # brace expansion: dir1 ... dir15
mkdir -p Linux/dirtest1/dirtest2 # create missing parents too
mkdir -m 777 mydir         # set permissions at creation
mkdir -v newdir            # verbose confirmation
mkdir /Linux/example      # create at a specific path
mkdir $USER               # environment variables work in names
```

Without `-p`, a missing parent makes the command fail; `-p` also tolerates directories that already exist.

2. Creating and Viewing Files: touch, cat, head, tail, echo, wc

2.1 touch — create empty files, update timestamps

A file made by touch is empty; on existing files it updates timestamps.

```
touch [options] file_name
touch File1                # one empty file
touch Doc1 Doc2 Doc3       # several at once
touch -a fileName          # access time only
touch -m fileName          # modification time only
touch -c fileName          # update only; do NOT create if missing
touch -r ref target        # copy timestamps from ref file
touch -t YYMMDDHHMM file   # explicit stamp
touch -d "17 Mar 2023" Geek.txt
```

Alternatives for an empty file: `cat > empty.txt` (type, then Ctrl+D), the `>` operator alone (`> empty.txt`), or an editor (`vim empty.txt`, then `:wq`).

2.2 cat — view, concatenate, create

Views contents, joins files, creates via redirection, appends.

```
cat [OPTION] [FILE]
cat notes.txt                # view one file
cat file1 file2              # view two files back to back
cat -n file2                 # with line numbers
cat > newfile                 # create/overwrite (type, Ctrl+D to save)
cat file1 file2 > merged.txt # merge into one file
cat file1 >> file2            # append file1 to the end of file2
cat -s filename              # squeeze repeated blank lines
cat -E filename              # show $ at each line end
cat -A filename              # -vET combined: visible non-printing chars, $ ends, ^I tabs
cat -- "-dashfile"          # file whose name starts with -
cat filename | more          # page through long output
cat *.txt                   # every .txt file in the folder
tac file2                   # reverse of cat: last line first
```

`>` overwrites, `>>` appends.

2.3 head — first lines

Prints the first 10 lines by default; previews logs, configs, CSVs without opening them.

```
head [options] [file(s)]
head /etc/passwd
head -n 3 /etc/hosts         # first 3 lines
head -c 100 data.csv         # first 100 bytes
head state.txt capital.txt   # headers ==> file <== before each file
head -q a.txt b.txt          # quiet: no file headers
head -v state.txt            # always print the header
ls -t | head -n 3            # three most recently modified files
head -n 20 f | tail -n 10    # lines 11--20 of the file
```

2.4 tail — last lines

Complement of head: last 10 lines by default; `-f` follows a growing log live.

```
tail [OPTION]... [FILE]...
tail /var/log/syslog
tail -n 20 /var/log/auth.log # or: tail -20 file
tail +25 state.txt           # from line 25 to end of file (tail-only form)
```

```
tail -c 200 notes.txt          # last 200 bytes
tail -f /var/log/syslog        # follow: new lines appear as written (Ctrl+C to stop)
tail -F app.log                # like -f, reopens the file after rotation
tail -n 7 state.txt | sort -r  # pipe into other filters
cat state.txt | head -n 20 | tail -n 5 > list.txt
```

Option	Meaning (head / tail)
-n NUM	First / last NUM lines (default 10).
-c NUM	First / last NUM bytes.
-q	No file headers with multiple files.
-v	Always print file headers.
-f / -F	Tail only: follow the file live; -F survives log rotation.
+n	Tail only: start printing from line n.

2.5 echo — print text

Shell builtin for messages, variables, and writing text into files.

```
echo [option] [string]
echo "Geeks for Geeks"
echo $HOME          # value of a variable
echo *              # file list, like ls
echo "Welcome" > out.txt  # write to file (overwrite)
echo "more" >> out.txt    # append to file
echo -n "no trailing newline"
echo -e "a\tb\nc"      # interpret escapes: \t tab, \n newline
echo -e "x\bc"         # \b backspace, \c stop output, \r carriage return, \a bell
```

2.6 wc — count lines, words, bytes

```
wc file.txt          # lines, words, bytes of the file
wc -l file.txt        # line count only
wc -w file.txt        # word count only
wc -c file.txt        # byte count
```

3. Copy, Move, Delete: cp, mv, rm

3.1 cp — copy files and directories

Needs at least two arguments; plain cp overwrites without warning.

```
cp source_file destination
cp a.txt b.txt          # copy file (overwrites b.txt silently)
cp a.txt b.txt c.txt new/  # several files into a directory
cp -R src_dir dest_dir   # recursive: copy a whole directory
cp -r geeksforgeeks gfg
cp -i a.txt b.txt        # interactive: ask before overwriting
cp -f a.txt b.txt        # force: overwrite even write-protected files
cp -n a.txt b.txt        # no-clobber: never overwrite
cp -p a.txt c.txt        # preserve timestamps and ownership
cp -v a.txt backup/      # verbose progress
cp -u a.txt backup/      # copy only if source is newer
cp *.txt Folder1         # wildcard: all .txt files
```

If `dest_dir` does not exist, `cp -R src dest` creates it as a copy of `src`; if it exists, `src` lands inside it as a subdirectory.

3.2 mv — move and rename

One command for renaming and relocating; on the same filesystem a rename keeps the timestamp and duplicates no data.

```
mv [options] source... destination
mv jayesh_gfg geeksforgeeks  # rename a file
mv jkj new_gfg               # rename a directory
mv geeksforgeeks /home/user/jkj/  # move into a directory
mv gfg_1 gfg_2 /home/user/jkj/    # move several files at once
mv -i old new                  # ask before overwriting (y/n)
mv -f old new                  # force overwrite, even write-protected
```

```
mv -n old new          # no-clobber: keep the existing file
mv -b old new          # back up the overwritten file with ~ suffix
```

3.3 rm — remove files and directories

Silent on success and unrecoverable: deleted files cannot be restored.

```
rm [OPTION]... FILE...
rm a.txt                # one file
rm b.txt c.txt          # several files
rm -i d.txt             # ask before each deletion
rm -f e.txt             # force: remove write-protected files without asking
rm -r dir/              # recursive: delete a directory tree (same as -R)
rm -- -file.txt         # delete a file whose name starts with -
```

rm * refuses directories (cannot remove 'B': Is a directory); rm -r * deletes everything inside. -f overrides file protection but not write-protected directories.

4. Help: man

4.1 man — read manual pages

man (manual) shows full documentation: usage, options, examples.

```
man [option] [command]
man ls                # manual page for ls
man 2 intro           # section 2 (system calls) entry for intro
man -f ls             # which sections document this command (= whatis)
man -a intro          # all matching pages in succession
man -k cd             # search all pages for "cd" (= apropos)
man -w ls             # path of the manual file on disk
man -I printf         # case-sensitive search
man vmstat            # manuals exist for commands like vmstat too
```

Navigate an open page with Spacebar (next page), Enter (next line), B (back a page), Q (quit). Pages have sections DESCRIPTION, OPTIONS, EXAMPLES, SEE ALSO; section 1 = shell commands, 2 = system calls, 3 = library calls, 5 = file formats, 8 = admin commands. Lighter alternatives: `whatis cmd` (one-line summary), `apropos keyword` (search descriptions), `tlldr cmd` (short examples), `info cmd` (hyperlinked docs), `man2html` (GUI-readable HTML).

5. Text Processing: cut, paste, sort, grep, locate

5.1 cut — slice columns out of lines

Cuts by byte, character, or delimited field; an option is mandatory (bare cut file errors). With no FILE it reads stdin.

```
cut OPTION... [FILE]
cut -b 1,2,3 state.txt # bytes 1, 2 and 3 of each line
cut -b 1-3,5-7 state.txt # byte ranges
cut -b 1- state.txt    # from byte 1 to end of line
cut -c 2,5,7 state.txt # characters 2, 5, 7
cut -c 1-7 state.txt   # first 7 characters
cut -d " " -f 1 state.txt # field 1 with space delimiter
cut -d " " -f 1-4 state.txt # fields 1 through 4
cut -f 1 state.txt     # default delimiter is TAB
cat state.txt | cut -d ' ' -f 1 | sort -r
```

Option	Meaning
-b LIST	Bytes, e.g. -b 1-3,7. Tabs count as one byte.
-c LIST	Characters, e.g. -c 2,5,7 or -c 1-7.
-d DELIM	Field delimiter instead of TAB, e.g. -d " " or -d ,.
-f LIST	Fields to keep, e.g. -f 1 or -f 1-4.
-n	Do not split multi-byte characters (with -b/-c).

5.2 paste — merge files side by side

Joins corresponding lines of files horizontally, TAB-separated by default; - reads stdin.

```

paste [OPTION]... [FILES]...
paste number state capital      # parallel merge, tab delimiter
paste -d ":" number state capital # custom delimiter (cycles if several given)
paste -s number state capital    # serial: each file becomes one long line
paste -s -d ":" number state capital
cat capital | paste - -          # join consecutive lines pairwise
cut -d' ' -f1 state | paste capital - # pipe shell data in as one side

```

5.3 sort — sort lines

Sorts alphabetically by default (uppercase before lowercase, numbers before letters); never modifies the input file unless `-o` is used.

```

sort [OPTION]... [FILE]...
sort file.txt                # alphabetical
sort -n file1.txt            # numeric (2 < 10 < 100)
sort -r example.txt          # reverse (Z to A)
sort -nr data.txt            # numeric, descending
sort -u names.txt            # sort and drop duplicates
sort -f mix.txt              # ignore case
sort -k3 employee_data.txt   # sort by 3rd field (blank = default separator)
sort -t, -k2 -n data.csv     # CSV: numeric sort on 2nd column
sort -c file.txt             # check: report if file is not sorted
sort -M months.txt           # month order (JAN..DEC)
sort -o sorted.txt file.txt   # write result to a file (safe: same name OK)
du -sh * | sort -hr          # human sizes, largest first

```

5.4 grep — search text by pattern

Prints lines matching a pattern (name from editor command `g/re/p`: globally search a regular expression and print).

```

grep [options] pattern [files]
grep "python" notes.txt      # lines containing python
grep -r "error" /var/log     # recursive search through a directory
grep -i "unix" geekfile.txt  # case-insensitive
grep -c "unix" geekfile.txt  # count of matching lines
grep -l "unix" *.txt         # only the matching file names
grep -w "unix" geekfile.txt  # whole word only
grep -o "unix" geekfile.txt  # only the matched text, not whole lines
grep -n "unix" geekfile.txt  # with line numbers
grep -v "unix" geekfile.txt  # invert: lines that do NOT match
grep "^unix" f               # lines starting with unix
grep "os$" f                 # lines ending with os
grep -e "a" -e "b" f         # several patterns
grep -f patterns.txt f       # patterns read from a file
grep -A1 learn geekfile.txt  # match + 1 line after (-B before, -C both)
grep -E "a|b" f              # extended regex
sudo grep -r "fail" /var/log  # sudo for protected logs

```

5.5 locate — find files by name

Searches a filename database instead of the disk, so it is faster than `find` but misses files created after the last database update and may list deleted ones.

```

locate [OPTION]... PATTERN...
locate sample.txt            # files named sample.txt
locate "*.html" -n 20        # first 20 matches
locate -c "*.txt"            # count matches only
locate -i *SAMPLE.txt*       # case-insensitive
locate -i -0 *sample*        # NUL-separated output (for xargs -0)

```

5.6 Piping text tools together

```

cat state.txt | head -n 20 | tail -n 5 > list.txt # lines 16-20 into a file
cat state.txt | cut -d ' ' -f 1 | sort -r         # first words, reverse-sorted
tail -n 7 state.txt | sort -r                     # last 7 lines, reversed
ls -t | head -n 3 | sort                          # 3 newest files, alphabetical
tar tvf file.tar | grep "pattern"                 # search inside an archive listing

```

```
du -sh /home/user/* | sort -hr
```

```
# disk hogs, largest first
```

6. Permissions and Privilege: chmod, sudo

6.1 Reading permissions

`ls -l` shows e.g. `-rw-r--r--`: first character is the type (`-` file, `d` directory), then owner/group/others triples of `r` read (4), `w` write (2), `x` execute (1). Checks run in order owner, then group, then others – exactly one triple applies. On directories, `r` lists entries, `w` creates/deletes entries, `x` allows `cd` and `ls -l` into it.

6.2 chmod — change mode

```
chmod [OPTIONS] MODE FILE
chmod 755 script.sh      # rwxr-xr-x: standard executable
chmod 644 file           # rw-r--r--: standard data file
chmod 600 secret.txt     # owner read/write only
chmod 700 private/       # owner full access only
chmod 777 file           # everyone everything (avoid)
chmod +x script.sh       # add execute for everyone
chmod u+x file           # add execute for owner
chmod g-w file           # remove write from group
chmod o-r file           # remove read from others
chmod a+r file           # read for all
chmod u=rwx,g=rx,o= file # exact per-class setting
chmod ug+rwx example.txt
chmod -R 755 public_html/ # recursive over a directory tree
```

Value	Meaning
4 / 2 / 1	read / write / execute; add per class, e.g. <code>rwx</code> = 4+2+1 = 7.
755	Owner <code>rwx</code> , group/others <code>r-x</code> . Scripts, public dirs.
644	Owner <code>rw-</code> , rest <code>r--</code> . Ordinary files.
600	Owner <code>rw-</code> only. Secrets.
700	Owner <code>rwx</code> only. Private dirs.
u g o a	Symbolic classes: user owner, group, others, all; <code>+</code> <code>-</code> = grant, remove, set.
Special bits	SUID (run as file owner), SGID (run as group / inherit group in dirs), sticky bit (only owners delete their files, e.g. <code>/tmp</code>).

Ownership itself changes with `chown user:group file` and `chgrp group file`.

6.3 sudo — run with elevated privilege

Prefix for commands only superusers may run (the “run as administrator” of Linux). Users need an entry in `/etc/sudoers`; always edit it with `visudo`, never a plain editor. Authentication uses the user’s own password.

```
sudo COMMAND          # run one command as root
sudo apt update
sudo nano /etc/hostname
sudo -u www-data whoami # run as another user
sudo -i               # interactive root shell
sudo -l               # list your allowed commands
sudo -v               # refresh the password timestamp
sudo -k               # invalidate it (next sudo asks again)
sudo -V               # version
```

Typical `sudoers` lines: `user1 ALL=(ALL) ALL` (full access), `user2 ALL=(ALL) NOPASSWD: ALL` (no password), `%admin ALL=(ALL) ALL` (whole group), `user3 ALL=/sbin/shutdown` (one command only).

7. Archives and Compression: tar, zip, gzip, bzip2, xz

7.1 tar — tape archive

Bundles files (optionally compressed); preserves permissions and directory structure.

```
tar [options] [archive-file] [files...]
tar cvf file.tar *.c      # create uncompressed archive
tar xvf file.tar          # extract
tar tvf file.tar          # list contents (verbose table)
tar tf file.tar            # list names only
```

```

tar cvzf file.tar.gz *.c          # create, gzip-compressed
tar xvzf file.tar.gz              # extract gzip archive
tar cvfj file.tar.tbz example.cpp # create, bzip2-compressed (smaller, slower)
tar xvf file.tar -C /dest/dir      # extract into a directory
tar xvf file.tar "fileA" "fileB"  # extract selected members
tar rvf file.tar *.c              # append to an existing archive
tar tvf file.tar --wildcards '*.png' # list only PNG members
tar tvf file.tar | grep "pattern"  # search an archive listing
tar czf file.tar | wc -c          # size of the archive in bytes

```

Flag	Meaning
-c / -x / -t	Create / extract / list.
-f FILE	Archive file name (must precede the name).
-v	Verbose file listing.
-z / -j / -J	Compress with gzip (.tar.gz) / bzip2 (.tar.bz2) / xz (.tar.xz).
-C DIR	Extract into DIR.
-r / -u / -A	Append files / add newer files / merge whole archives.
-W	Verify archive integrity.

7.2 zip, gzip, bzip2, xz

```

zip [options] zipfile files...    # zip keeps originals, tar-like tools replace them
zip -r myfiles.zip Documents/     # recurse a directory
zip -e -r secure.zip sensitive/   # password-encrypted
zip -9 -r a.zip files/            # max compression (0..9)
zip -u a.zip newfile              # update archive / -d delete / -x exclude pattern
unzip myfiles.zip                 # extract a zip archive
gzip bigfile                      # in-place -> bigfile.gz (original replaced)
bzip2 bigfile                     # in-place -> bigfile.bz2 (tighter, slower than gzip)
xz bigfile                        # in-place -> bigfile.xz (tightest, slowest)

```

Rule of thumb: zip for portable/Windows-compatible archives, tar -czf for Linux backups, bzip2/xz when size matters more than speed.

8. Disks, Memory, System: df, du, free, top, vmstat, uname

8.1 df — disk free

Reports filesystem usage (from /proc/mounts or /etc/mtab); default sizes are 1K blocks.

```

df [options] [filesystems]
df                                # all mounted filesystems
df jayesh.txt                     # filesystem holding that file
df -h                            # human-readable, powers of 1024 (use this by default)
df -H                            # human-readable, powers of 1000
df -T                            # include filesystem type (e.g. ext4)
df -i                            # inode usage instead of blocks
df -a                            # include pseudo filesystems
df -x tmpfs                      # exclude a filesystem type
df --total                       # grand-total line

```

Columns: Filesystem, Size, Used, Avail, Use%, Mounted on.

8.2 du — disk usage

Sizes files and directories recursively; defaults to the current directory.

```

du [options] [directory/file]
du /home/user/test
du -h /home/user/test            # human-readable
du -sh /home/user/test          # summary: one total line (the classic form)
du -a -h dir                    # all files, not just directories
du -c -h dir                    # plus a grand total line
du -d 1 dir                     # limit depth to 1 level
du --time -h dir                # with last-modification timestamps
du -sh /home/user/* | sort -hr  # find the space hogs

```

8.3 free — memory overview

Total/used/free RAM and swap; plain `free` prints kilobytes.

```
free [OPTION]
free -h          # human-readable (use this by default)
free -m          # megabytes (-b bytes, -k kilobytes, -g gigabytes)
free -t          # extra totals line
free -s 3 -c 3   # repeat 3 times, every 3 seconds
```

Columns: total, used, free, shared (tmpfs), buffers, cached, available.

8.4 top — live process monitor

Full-screen, real-time view of load, CPU, memory, and the process list (q to quit).

```
top
top -d 5          # refresh every 5 seconds
top -u www-data   # only one user's processes
top -p 1234        # only PID 1234
top -b -n 1 > status.txt # batch mode: one snapshot to a file
```

Header lines: uptime + users + load averages (1/5/15 min), task counts, %Cpu split (us user, sy system, ni niced, id idle, wa I/O wait), RAM and swap summaries. Per-process columns: PID, USER, PR, NI (nice value), VIRT, RES, SHR, S (R running, S sleeping, Z zombie), %CPU, %MEM, TIME+, COMMAND.

8.5 vmstat — virtual memory statistics

One-line snapshots of processes, memory, swap, I/O, and CPU; the first report averages since boot, later ones cover the sampling interval.

```
vmstat [options] [delay [count]]
vmstat          # single snapshot
vmstat 1 5      # every 1 second, 5 times
vmstat -a       # active vs inactive memory
vmstat -f       # fork count since boot
vmstat -m       # slab info (kernel-dependent)
vmstat -s       # event counters and memory statistics table
vmstat -d       # disk statistics
vmstat -t 1 3   # with timestamps
vmstat -S M 1 3 # memory figures in megabytes
vmstat -n       # header printed only once
```

Watch `free` (idle memory) and `si/so` (swap in/out per second): nonzero `so` under load means the box is paging.

8.6 uname — system information

`uname` = Unix name: kernel and hardware identity.

```
uname [OPTIONs]
uname -a    # everything: kernel, hostname, release, version, arch, OS
uname -s    # kernel name (Linux)
uname -n    # hostname
uname -r    # kernel release
uname -v    # kernel version / build date
uname -m    # machine hardware (e.g. x86_64)
uname -p    # processor type
uname -i    # hardware platform
uname -o    # operating system (GNU/Linux)
```

9. Processes: fork, exec, ps, kill, nice

9.1 fork() — create a process

System call that duplicates the running process. The original is the parent, the copy is the child: new PID, copied memory, same code, environment, and open files; both continue after the `fork()` call.

```
pid_t fork(void); /* parent gets child PID (>0), child gets 0, -1 on failure */
```

Used for child processes, parallel tasks, servers handling many clients, and every shell command launch.

9.2 exec() family — run another program

Replaces the current program image with a new one: same PID, new memory. The standard pattern is `fork()` then `exec()`: the shell forks, the child execs `ls`, the parent waits.

```
execl("/bin/ls", "ls", "-l", NULL);
```

Function	Style
<code>execl()</code> / <code>execv()</code>	Argument list / argument array.
<code>execlp()</code> / <code>execvp()</code>	As above but search PATH.
<code>execle()</code> / <code>execve()</code>	Custom environment; <code>execve</code> is the real system call.

`fork()` creates (new PID, returns twice) while `exec()` replaces (same PID, returns only on failure).

9.3 ps — process status

```
ps                # your processes on this terminal (PID TTY TIME CMD)
ps -e             # every process (same as ps -A)
ps -ef            # full details: UID PID PPID C STIME TTY TIME CMD
ps -u ann         # one user's processes
ps aux            # BSD form: USER PID %CPU %MEM COMMAND (most-used form)
ps -l             # long form incl. NI nice-value column
```

Key fields: PID, PPID (parent), UID/USER, TTY, %CPU, %MEM, CMD.

9.4 kill — signal processes

Sends signals, not just death: default is graceful SIGTERM (15).

```
kill PID          # SIGTERM: polite shutdown (same as kill -15 PID)
kill -9 2000       # SIGKILL: force kill, cannot be caught
kill -STOP 2000    # pause (resume with kill -CONT 2000)
kill -2 2000       # SIGINT: like Ctrl+C
kill 1001 1002 1003 # several PIDs at once
pkill firefox      # kill by name
killall firefox    # kill all processes with this name
```

Signal	Number	Purpose
SIGHUP	1	Reload configuration.
SIGINT	2	Interrupt (Ctrl+C).
SIGTERM	15	Graceful termination (default).
SIGKILL	9	Force kill, uncatchable.
SIGSTOP / SIGCONT	19 / 18	Pause / resume.

9.5 nice — process priority

Nice values run -20 (highest CPU priority) through 0 (default) to 19 (lowest); only root can set negative values. Lower nice = more CPU.

```
nice -n 10 firefox # start with low priority
nice ls            # start at default priority
sudo nice -n -10 firefox # start with highest priority (root only)
renice 5 -p 2200   # change a RUNNING process (PID 2200)
ps -l             # check the NI column
```

10. Master Quick Reference

Command	Purpose / flagship syntax
<code>pwd</code>	Where am I: <code>pwd [-L -P]</code> .
<code>cd</code>	Go elsewhere: <code>cd [dir]</code> ; <code>cd ~</code> ; <code>cd ..</code> ; <code>cd -</code> ; <code>cd /</code> .
<code>ls</code>	List: <code>ls -lh</code> , <code>ls -a</code> , <code>ls -t</code> , <code>ls -R</code> .
<code>mkdir</code>	Create dirs: <code>mkdir -p a/b/c</code> , <code>-m 777</code> , <code>-v</code> .
<code>touch</code>	Empty file / timestamps: <code>touch f</code> , <code>-a</code> <code>-m</code> <code>-c</code> <code>-r</code> <code>-t</code> <code>-d</code> .
<code>cat</code>	View/join/create: <code>cat f</code> , <code>-n</code> <code>-s</code> <code>-E</code> <code>-A</code> , <code>>/>></code> .
<code>head / tail</code>	First / last 10 lines: <code>-n</code> , <code>-c</code> ; <code>tail -f</code> .
<code>echo</code>	Print text/variables: <code>echo -n</code> , <code>-e</code> , <code>>/>></code> .

Command	Purpose / flagship syntax
wc	Count: <code>wc -l -w -c file</code> .
cp	Copy: <code>cp -R -i -f -n -p -u -v</code> .
mv	Move/rename: <code>mv -i -f -n -b</code> .
rm	Delete (unrecoverable): <code>rm -i -f -r, rm -- -f</code> .
man	Manuals: <code>man cmd, -f -k -a -w</code> , navigate Space/B/Q.
cut	Slice lines: <code>-b -c -d -f</code> .
paste	Merge side by side: <code>-d, -s, - = stdin</code> .
sort	Sort lines: <code>-n -r -u -f -k -t -o</code> .
grep	Search: <code>-i -v -c -n -l -r -w -o -A/-B/-C -E</code> .
locate	Find by name via DB: <code>-i -c, -n 20</code> .
chmod	Permissions: 755 644 600 700, u+x, -R.
chown/chgrp	Owner/group: <code>chown user:group f</code> .
sudo	As root: <code>sudo cmd, -u -i -l -k</code> ; edit via visudo.
tar	Archive: <code>-c -x -t -f -v -z -j -J -C</code> .
zip/unzip	Portable archives: <code>zip -r -e -9</code> .
gzip/bzip2/xz	In-place compress: <code>.gz fast, .bz2 tighter, .xz tightest</code> .
df	Filesystem free space: <code>df -h -T -i --total</code> .
du	Directory sizes: <code>du -sh, -a -c -d N --time</code> .
free	RAM/swap: <code>free -h -m -t -s N</code> .
top	Live monitor: <code>-d -u -p -b -n</code> .
vmstat	VM stats: <code>vmstat [delay [count]], -a -s -d -S M</code> .
uname	System identity: <code>-a -s -n -r -v -m -p -i -o</code> .
fork/exec	Create process / replace program: <code>fork(); execl(...)</code> .
ps	List processes: <code>ps aux, ps -ef, ps -l</code> .
kill	Signal: <code>kill [-9 -15] PID, pkill/killall name</code> .
nice/renice	Priority -20..19: <code>nice -n 10 cmd, renice 5 -p PID</code> .

Source Scope

Compiled from every file in `linux-cmds/`: all 27 .docx notes (`pwd/cd/ls/mkdir/touch/cat/mv/cp/rm/man/echo/cut/locate/ls/head/ps-exec-kill-nice`), the `Linux_Commands_Guide.pptx` deck (`df, du, tar, zip, uname, chmod, head, tail, sort, grep, sudo, top, free, vmstat`), the `pwd mkdir Commands.pptx` deck (`pwd -L/-P, mkdir -p/-m/-v, brace expansion`), and `Basic Shell Commands in Linux.pdf` (command categories, `wc, chown/chgrp, ps/kill` examples). Terminology, syntax forms, and option sets follow the sources; overlapping coverage was merged into single per-command sections. Peripheral items mentioned only in the overview PDF (networking: `ping, wget, curl, ssh, scp, ftp`; packages: `dnf, yum, apt-get`; jobs: `bg, killall`) are listed here for completeness but are outside this cheatsheet's command set.

Study principle: understand the mechanism first, then memorize the exam wording.